Prakses uzdevums: „Spēle ar savu serveri”

Grupa P25-2 · 12.10.–20.10.2026 · BL 228 · Individuāls darbs, apmēram 40 stundas

Īsumā. Katrs izstrādā savu 2D spēli Python valodā ar Pygame. Spēle sazinās ar paša rakstītu REST API serveri (FastAPI vai Flask + SQLite), kas glabā rezultātus datubāzē, un atdod HTML/CSS mājaslapu ar līderu tabulu. Kodam jābūt sakārtotam un saprotamam (PEP-8 stils), README apraksta, kā visu palaist. 19.10. prezentācija un iesniegšana, 20.10. aizstāvēšana.

1. Kas jāizstrādā

1.1. Spēle (Pygame)

Žanru izvēlies no saraksta: Snake, Breakout, Space shooter, Platformer, Tetris, 2048 vai Sokoban, Tower defense, vienkāršs Roguelike. Cits žanrs tikai pēc saskaņošanas ar skolotāju.

- Galvenā izvēlne, kur spēlētājs ievada savu vārdu.
- Vismaz 3 līmeņi vai pieaugoša grūtība.
- Punktu skaitīšana, pauze, spēles beigu ekrāns.
- **Divas unikālas mehānikas**, kuru nav parastajā žanra versijā. Tās pirmajā dienā ieraksta kopīgajā Google tabulā, skolotājs tās apstiprina, un grupā tās nedrīkst atkārtoties.
- **Spēles loģika ir atdalīta no zīmēšanas.** Kustība, sadursmes, punkti un līmeņi ir klasēs mapē `game/`, kur nav `import pygame`. Mape `client/` tikai nolasa taustiņus un zīmē.
- Spēle nesasalst un nenokrīt, ja serveris nav pieejams (paziņo spēlētājam).

1.2. REST API serveris

FastAPI (ieteicams) vai Flask, datubāze SQLite. Minimālais endpointu saraksts:

Metode	Adrese	Ko dara	Veiksmīga atbilde
POST	/scores	Saglabā rezultātu (vārds un punkti)	201
GET	/scores/top	Top-10 rezultāti	200
GET	/players/{name}/stats	Spēlētāja statistika (spēles, labākais)	200 / 404

- Pareizi statusa kodi: 200, 201, 400 vai 422 (slikti dati), 404 (nav atrasts).
- Ievaddatu validācija: tukšs JSON, negatīvi punkti vai pārāk garš vārds netiek ierakstīts datubāzē.
- SQL vaicājumi ar parametriem, nevis virkņu salīmēšanu.

1.3. Mājaslapa (HTML/CSS)

Lapas atdod tas pats serveris ar Jinja veidnēm, JavaScript nav vajadzīgs:

- `GET /` — līderu tabula (top-10).
- `GET /players/{name}` — spēlētāja profils ar statistiku.
- Lapa ar spēles aprakstu un noteikumiem.
- Kopīga veidne (`base.html`) un savs CSS fails.

1.4. Koda kvalitāte un rīki

Prasība	Mērķis	Kā pārbaudīt
PEP-8 stils	Saprotami nosaukumi, 4 atstarpes, sakārtots kods	Koda apskate
Docstringi un tipu anotācijas	Visām publiskajām funkcijām un klasēm	Koda apskate
Git	Commiti katru dienu, jēgpilnas ziņas	git log
README.md	Palaišanas instrukcija, endpointu tabula, ekrānuzņēmums	Atvērt repozitorijā

Testēšana: automātiskie testi (pytest) nav obligāti. Kā pārbaudīt, ka spēle un serveris strādā, izvēlies pats (spēlējot, ar `/docs` lapu, ar print, ar saviem testiem), bet iesniegtajam projektam jāstrādā bez kļūdām.

2. Tehnoloģijas, kas jāizmanto

Joma	Rīks
Valoda	Python 3.11 vai jaunāks, virtuālā vide (venv), requirements.txt
Spēle	pygame
Serveris	FastAPI + uvicorn (vai Flask), Pydantic validācijai
Datubāze	SQLite (sqlite3 vai SQLAlchemy)
Klients ↔ serveris	requests, JSON
Mājaslapa	Jinja2 veidnes, HTML, CSS
Versiju kontrole	Git, privāts GitHub repozitorijs

3. Obligātā projekta struktūra

```
spele/
├── game/              # spēles noteikumi, BEZ pygame
│   ├── __init__.py
│   └── ...
├── client/            # pygame: ievade, zīmēšana, api_client.py
│   └── main.py
├── api/               # FastAPI serveris
│   ├── main.py
│   ├── db.py
│   ├── templates/     # Jinja HTML veidnes
│   └── static/        # CSS
├── Dizains.docx       # spēles apraksts (Word)
├── README.md
├── requirements.txt
└── .gitignore         # .venv/, __pycache__/, *.db
```

4. Spēles apraksts (Word fails)

Viena lappuse Word failā (`Dizains.docx`) repozitorija saknē, jāuzraksta pirmajā dienā. Nosaukumu, žanru un 2 mehānikas ieraksti arī kopīgajā Google tabulā (saite Mykoob); līdz 12:50 tām jābūt apstiprinātām. Aizstāvēšanā spēli salīdzina ar šo dokumentu.

– Nosaukums un žanrs.
– Mērķis: kā spēlētājs uzvar un kā zaudē.
– Vadība: kuri taustiņi ko dara.
– Pamata cikls: darbība → rezultāts → atlīdzība → izaicinājums.
– Divas unikālās mehānikas, katra vienā teikumā.
– Līmeņi: kā aug grūtība.
– Punktu sistēma: par ko un cik.
– Ko spēle saglabā serverī un ielādē.
– Ekrāna skice (zīmējums uz papīra, nofotografēts).

5. Grafiks un kontrolpunkti

Diena	Tēma	Kontrolpunkts dienas beigās
P 12.10	Dizains, repozitorijs, spēles cikls	Mehānikas apstiprinātas tabulā (12:50); objekts kustas uz ekrāna
O 13.10	Spēles loģika, izvēlne, pauze	Spēlējams kodols: var spēlēt, zaudēt, redz punktus
T 14.10	REST API, SQLite	Starpdemo: spēle darbojas; 3 endpointi ar datubāzi
C 15.10	Spēle ↔ API, HTML/CSS lapas	Rezultāts nonāk serverī un līderu tabulā; abas mehānikas
Pk 16.10	Pabeigšana, README (08:20–11:30)	README repozitorijā; abas mehānikas strādā
P 19.10	Prezentācijas, pabeigšana	**08:20** prezentācija Moodle; **13:35** tags v1.0 GitHub
O 20.10	Aizstāvēšana pie sava datora	Spēle un serveris strādā; 5 jautājumi par kodu

6. Ikdienas kārtība

- Mācības: P–C 08:20–15:10, piektdien 08:20–11:30, 19. un 20.10. 08:20–13:35. No rīta īsa teorija, pēc tam patstāvīgs darbs.
- Katru dienu vismaz 3 commiti un push līdz 15:10.
- **Dienas noslēgums 14:40–15:10 (piektdien 11:00):** katrs 1 minūtē pastāsta (un parāda ekrānā): 1) ko šodien izdarīju, 2) ko darīšu rīt, 3) kur iestrēgu.

7. Prezentācija (19.10.)

PowerPoint (vai PDF) ar 5–6 slaidiem un ekrānuzņēmumiem, apmēram 4 minūtes. Spēli klasei dzīvajā nerāda. Prezentāciju ielādē Moodle **līdz 19.10. plkst. 08:20**, skolotājs tās atver pēc kārtas no sava datora.

1. Spēles nosaukums un autors.
2. Kā spēlē (ekrānuzņēmums).
3. Abas mehānikas: kas tās ir (ekrānuzņēmumi).
4. Serveris un mājaslapa (ekrānuzņēmums no /docs un līderu tabulas).
5. Kas bija grūtākais un kā to atrisināju.

Kritērijs	Punkti
Spēle un abas mehānikas skaidri parādītas	4
Paskaidrots, kā spēle, serveris un datubāze strādā kopā	3
Grūtākā vieta un kā to atrisināja	1
Laiks ievērots, saprotami izstāstīts	2

8. Iesniegšana (19.10. līdz 13:35)

- Privāts GitHub repozitorijs, skolotājs pievienots kā collaborator: **chillinxol**.
- Gala versija atzīmēta ar tagu: `git tag v1.0` un `git push --tags`. Vēlākus commitus nevērtē.
- Repozitorija saiti iesniedz Mykoob pie šī uzdevuma.

9. Vērtēšana

Daļa	Punkti
Spēles funkcionalitāte un unikālās mehānikas	35
REST API	20
Tīmekļa lapa	10
Koda kvalitāte (struktūra, loģikas atdalīšana, PEP-8 stils)	10
Git vēsture un README	15
Prezentācija	10
Kopā	**100**

Aizstāvēšana un koeficients. 20.10. skolotājs uzdod 5 jautājumus par tavu kodu (pie tava datora: palaid spēli un serveri, tad, piemēram: parādi, kur apstrādā sadursmi; kāpēc endpoints atgriež 201; kas notiek, ja serveris ir izslēgts; dzīvā izmaiņa kodā). Iegūtos punktus reizina ar koeficientu: 5 pareizas atbildes → ×1.0, 4 → ×0.9, 3 → ×0.75, 2 vai mazāk → ×0.5. Piemērs: 80 punkti un 3 pareizas atbildes = 60 punkti.

10. Mākslīgā intelekta lietošana

AI rīkus drīkst lietot: lai paskaidrotu kļūdas, izskaidrotu bibliotēkas, palīdzētu atrast kļūdas un pārskatītu kodu. **Noteikums: par katru koda rindiņu atbildi tu pats.** Ja ielīmē kodu, kuru nevari paskaidrot vai aizstāvēšanā izmainīt, tas atspoguļosies koeficientā. Projekts, kas parādās vienā commitā pēdējā vakarā, rada jautājumus par tā izcelsmi.

11. Papildu punkti (līdz +10, ne vairāk par 100 kopā)

- Spēlētāju reģistrācija un pieteikšanās: parole datubāzē kā hash, tokens, POST /scores tikai ar tokenu.
- Spēles saglabāšana serverī (PUT un GET /saves).
- Gājienu multiplayer caur API.

- Līmeņu redaktors ar līmeņu saglabāšanu serverī.
- Sasniegumu sistēma (glabājas serverī, redzama profilā).
- Automātiskie testi ar pytest.

12. Pārbaudes saraksts pirms iesniegšanas

□ Spēles apraksts (Word) ir repozitorijā, spēle atbilst tam.
□ Izvēlne, 3 līmeņi, pauze, beigu ekrāns, abas mehānikas.
□ game/ mapē nav import pygame.
□ Visi minimālie endpointi strādā, statusa kodi pareizi.
□ Līderu tabula un profils atveras pārlūkā.
□ Spēle un serveris strādā bez kļūdām.
□ README ar palaišanas instrukciju un endpointu tabulu.
□ Tags v1.0 ir GitHub, skolotājs redz repozitoriju.